

THE IMPORTS THAT SHOW UP IN EVERY REAL PROJECT

The Python Stack for AI Engineers

Forty packages worth knowing, what each one is actually for, and the six you could ship a product with tomorrow.

The Stack

LAYERS

The ecosystem looks enormous until you see it as six layers. You pick roughly one package per layer, and the rest is noise until you need it.

APPLICATION	fastapi · pydantic · streamlit
ORCHESTRATION	langgraph · llamaindex · crewai
MODEL ACCESS	anthropic · openai · litellm · vllm
RETRIEVAL	sentence-transformers · qdrant · pgvector
MODELLING	pytorch · transformers · scikit-learn
DATA	pandas · polars · numpy · pyarrow

six layers, and most teams only need one package from each



Pick what your team already runs



Then what has the best docs



Then what is actively maintained



Then whichever is smallest

Adding a package is permanent: it becomes something to upgrade, audit and explain to the next engineer.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Arrays and Frames

FOUNDATION

Everything sits on these. If you are fast here, every other layer gets easier.

PACKAGE	USE IT FOR
numpy	N-dimensional arrays, the type everything else speaks
pandas	Tabular work, the default for anything under a few GB
polars	Same job, far faster, lazy execution, strict types
pyarrow	Columnar memory and Parquet, the interchange format
duckdb	SQL directly over Parquet and DataFrames, no server

```
import polars as pl
df = pl.scan_parquet('events/*.parquet') # lazy, no load yet
      .filter(pl.col('ts') > cutoff).group_by('user').len().collect()
```

⚡ polars is lazy and multi threaded by default

🗄 duckdb queries Parquet without loading it

Reach for polars or duckdb the moment pandas starts swapping. The rewrite is usually an afternoon and buys you a year.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Seeing the Data

VISUALS

You cannot clean what you have not looked at. These are worth ten minutes each.

PACKAGE	BEST AT
matplotlib	The base layer, total control, verbose
seaborn	Statistical plots in one line, great defaults
plotly	Interactive charts, hover and zoom, good for dashboards
altair	Declarative grammar, clean for exploration



Histogram every numeric column once



Correlation heatmap before feature work



Plot residuals, not just the score



Plot every metric against time

Learn seaborn for exploring and matplotlib for the one chart that ends up in the deck. Interactive libraries are for dashboards, not analysis.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Tabular Models

STILL WINNING

On tabular data these still beat deep learning, and they train in seconds on a laptop.

PACKAGE	ROLE
<code>scikit-learn</code>	The API everything else copies, plus every classic model
<code>xgboost</code>	Gradient boosting, the tabular competition winner
<code>lightgbm</code>	Faster on large datasets, handles categories natively
<code>catboost</code>	Best with high cardinality categorical features
<code>statsmodels</code>	When you need p values and confidence intervals

```
from sklearn.pipeline import Pipeline
# a Pipeline is what stops you leaking during CV
clf = Pipeline([('prep', prep), ('model', LGBMClassifier())])
```

Start every tabular problem with LightGBM and a Pipeline. Beat that baseline with numbers before anything deeper enters the conversation.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Training Frameworks

TENSORS

PyTorch has effectively won. The others exist for specific reasons, not as general alternatives.

PACKAGE	WHY IT EXISTS
<code>torch</code>	The default. Research and production both run on it
<code>lightning</code>	Removes the training loop boilerplate and the device juggling
<code>jax</code>	Functional, compiled, great for research at scale
<code>tensorflow</code>	Legacy systems and some mobile deployment paths
<code>onnxruntime</code>	Export once, run anywhere, without the training framework



`accelerate` handles the device placement



Mixed precision roughly halves memory



`torch.compile` is often a free speedup



Export to ONNX only when you must

Learn raw PyTorch first. Every abstraction on top of it makes sense afterwards, and none of them make sense before.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Pretrained and Fine Tuning

HUGGING FACE

The Hugging Face stack is how you get a model, adapt it, and keep it in memory without a data centre.

PACKAGE	JOB
<code>transformers</code>	Load and run almost any open model with one API
<code>datasets</code>	Stream and map large datasets without loading them
<code>tokenizers</code>	Fast tokenisation, the Rust backed implementation
<code>peft</code>	LoRA and QLoRA, fine tune a fraction of the weights
<code>trl</code>	SFT, DPO and reward modelling on top of transformers
<code>accelerate</code>	Same script on CPU, one GPU, or many, unchanged
<code>bitsandbytes</code>	4 and 8 bit quantisation so a big model fits

QLoRA with peft and bitsandbytes is how a 7B model gets fine tuned on a single consumer GPU. That combination is why open models became practical.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Running Models Yourself

INFERENCE

Training frameworks are slow at serving. These exist purely to make inference fast and cheap.

PACKAGE	BEST FOR
vllm	High throughput GPU serving, paged attention, batching
ollama	Local models with one command, great for development
llama.cpp	CPU and Apple silicon, quantised GGUF models
TGI	Hugging Face production server, solid defaults
sglang	Structured generation and fast multi turn serving

 vLLM is usually 10 to 20x faster than a naive loop

 ollama for the laptop, vLLM for the cluster

 Quantisation trades a little quality for a lot of memory

Self hosting only wins on cost at real volume. Below that, an API is cheaper once you count the engineering time.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Talking to Providers

SDKS

Thin clients around HTTP. The value is in retries, streaming and typed responses, not in the call itself.

PACKAGE	NOTE
<code>anthropic</code>	Claude SDK, streaming, tool use, prompt caching
<code>openai</code>	GPT SDK, the shape most other clients imitate
<code>litellm</code>	One interface over 100 providers, easy failover
<code>instructor</code>	Typed structured output validated with pydantic
<code>tenacity</code>	Retry with exponential backoff, not AI specific but essential

```
from anthropic import Anthropic
r = Anthropic().messages.create(model='claude-opus-4-5',
                                max_tokens=1024, messages=msgs)
```



Wrap every call in retry with backoff



Set an explicit timeout, the default is too long



Check the stop reason before parsing



`instructor` gives you typed output for free

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Wiring It Together

PIPELINES

These give you the control flow: retrieval, tool calls, retries and state. Useful when the graph gets real, overhead when it does not.

PACKAGE	STRENGTH
langchain	The largest set of integrations and loaders
langgraph	Explicit state machine, loops, retries, human in the loop
llamaindex	Strongest on indexing and retrieval strategies
haystack	Production pipelines with a clear component contract
dspy	Optimises the prompts for you instead of you tuning them



Start with no framework at all



Add one when you need loops and retries

If your flow is one retrieval and one model call, a plain function beats every framework here.

Frameworks are a bet on your flow getting complicated. Sometimes that bet is wrong.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Multi Step Systems

AGENTS

Everything here is a loop around a model with tools. They differ in how much structure they impose on that loop.

PACKAGE	SHAPE
langgraph	Graph of nodes, most control, most explicit
crewai	Role based crews, readable, opinionated
autogen	Conversational multi agent, strong research lineage
pydantic-ai	Typed, minimal, validation first
smolagents	Tiny, code writing agents, very little magic



Cap the iterations on every agent



Log every tool call and its arguments



Human approval for irreversible actions



Track cost per session, not per call

The framework is the easy part. Iteration limits, tool error handling and observability are what make an agent survive production.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Vectors and Reranking

RAG

The retrieval half of RAG. The embedding model matters more than the store, and reranking matters more than either.

PACKAGE	ROLE
<code>sentence-transformers</code>	Embeddings and cross encoder rerankers, local
<code>FlagEmbedding</code>	BGE models, strong open embeddings and rerankers
<code>faiss</code>	In process ANN index, fast, no server
<code>chromadb</code>	Simple local vector store, good for prototypes
<code>qdrant-client</code>	Production vector database with real filtering
<code>pgvector</code>	Vectors inside Postgres, one system instead of two
<code>rank-bm25</code>	Keyword scoring for hybrid search, tiny and useful



Pick the embedding model first



Then add a reranker, then tune the store

Adding a cross encoder reranker is usually a bigger quality jump than switching vector database.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Language Processing

MLP

Still needed around the model: cleaning, chunking, entity extraction and counting tokens before you pay for them.

PACKAGE	USE FOR
spacy	Fast production NER, POS, sentence splitting
nltk	Teaching and research, wide but slower
tiktoken	Count tokens before you send them
ftfy	Fixes mojibake and broken unicode in scraped text
rapidfuzz	Fuzzy string matching and deduplication, very fast
unstructured	Parse PDFs, HTML and docx into clean elements



unstructured handles the PDF mess for you



spacy NER redacts before data leaves

Counting tokens locally with tiktoken before the call is the cheapest cost control you can add to any LLM system.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Evaluation

MEASUREMENT

Without these you are shipping on vibes. Every one of them exists because eyeballing outputs does not scale.

PACKAGE	MEASURES
ragas	RAG specific: faithfulness, context precision and recall
deepeval	Pytest style LLM assertions in CI
promptfoo	Prompt and model comparison from a config file
evaluate	Classic metrics: BLEU, ROUGE, accuracy
giskard	Scans models for bias and robustness problems



Wire evaluation into CI on day one



200 examples minimum before you trust a number



Calibrate an LLM judge against human labels first

An evaluation set you built in an afternoon is worth more than any framework you adopt on top of it.

↓ [CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Seeing Production

OBSERVABILITY

An LLM call you cannot inspect is a bug you cannot fix. Tracing is not optional once real users arrive.

PACKAGE	GIVES YOU
langfuse	Traces, costs, prompt versions, self hostable
phoenix	Local tracing and evaluation, strong for debugging
opentelemetry	The vendor neutral standard underneath most of them
mlflow	Experiment tracking and a model registry
wandb	Training run tracking, sweeps, team dashboards



Attribute cost per feature,
not per app



Trace the whole chain, not
just the final call

Log the assembled prompt, the model version, every parameter and the response. Without those four you cannot explain any output later.

Add tracing before your first real user, not after your first incident.

↓ [CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Shipping It

BACKEND

The unglamorous half of AI engineering, and the half that decides whether anyone can actually use what you built.

PACKAGE	ROLE
<code>fastapi</code>	Async API framework, typed, automatic docs
<code>pydantic</code>	Validation and settings, the type layer everything uses
<code>uvicorn</code>	ASGI server that runs it
<code>celery</code>	Background jobs for anything slower than a request
<code>redis</code>	Cache, queue, rate limiter, session store
<code>httpx</code>	Async HTTP client with sane timeouts and retries



Set a timeout on every outbound call



Stream long responses, do not block



Rate limit per tenant, not globally



Add a health check before you deploy

An LLM call is slow and can fail. That makes it a background job far more often than teams expect.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Moving Data

ORCHESTRATION

Ingestion, embedding and reindexing are scheduled jobs. Treat them like data engineering, because they are.

PACKAGE	USE WHEN
prefect	Python native flows, lightest to adopt
dagster	Asset oriented, strong lineage and typing
airflow	The industry default, heavy but everywhere
dbt	SQL transformations with tests and documentation
great-expectations	Data quality assertions as a pipeline stage



Make every run idempotent



Alert on zero rows, not just on errors

Your RAG ingestion is an ETL job. Give it retries, idempotency and alerting on the day you build it, not after it silently fails.

A stale index fails quietly. Nobody gets paged, the answers just get worse.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Keeping It Sane

DEVELOPER

Not AI specific, and they will save you more time than any model choice.

TOOL	REPLACES
<code>uv</code>	<code>pip</code> , <code>venv</code> and <code>poetry</code> , ten to a hundred times faster
<code>ruff</code>	<code>flake8</code> , <code>isort</code> and <code>black</code> , in one fast binary
<code>pytest</code>	The only test runner worth learning
<code>mypy</code>	Catches the type bugs <code>pydantic</code> does not
<code>docker</code>	Works on my machine, permanently



Pin every version, an unpinned stack breaks silently



Separate dev and production dependencies



`uv` makes a clean install fast enough to do often

Switching to `uv` and `ruff` takes an hour and gives you back minutes every single day for the rest of the project.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The Minimal Stack

[START HERE](#)

If you learn only these six properly, you can build and ship most AI products end to end.

pandas
data

scikit-learn
baselines

pytorch
models

transformers
pretrained

fastapi
serving

pydantic
contracts

this covers roughly 90 percent of real AI engineering work

Depth in six packages beats a passing familiarity with forty. Every extra dependency is something you will one day have to debug at 2am.



Learn these six deeply first



Add the seventh only when they cannot cope

Every extra dependency is something you will one day debug at 2am.

↓ [CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

The Python Toolkit



pandas and polars



matplotlib



scikit-learn



pytorch



transformers and peft



vllm and ollama



anthropic and litellm



langgraph



sentence-transformers



ragas



langfuse



fastapi and pydantic

**Anyone can pip install forty things.
Engineers know which six
they actually need.**

Save this before your next project setup.

↓ [CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)